

Scrabble --- Two Computer game

Homework for ITP-368 GUIs

Goal

You are to make a Scrabble (-like) game that runs between two computers. If you really want to do some other game, ask me. (BattleShip works. Many board games can work. We will be starting from TicTacToe, so TicTacToe-like games are not an option.)

Requirements

- The game involved should have a GUI interface, something represented graphically. Do not just use the word exchange given to you in Chatter.
- Each user should get a view of the state of the game, but they should only be able to play their side / their turn from their computer. (duh)

Suggestions on coding

- I highly recommend you start with Chatter or the TTT2 from class. These already have the communication set up. All you have to do is modify what the user sees and supply the logic of the game. Oh, and you have to add messages between the two parts, so that each part knows what the other person plays.
- Design the messages early, the messages that communicate between the two sides of the game. The rest of the programming task tends to be just to implement what is in these messages.
- The goal is for this to work across 2 machines. But to test it, you can run it (as we will do in class) on a single machine. Just start the program twice, and select "host" on one and "client" on the other, with "localhost" as the IP of the host from the client side.
- Dragging tiles looks great, but clicking on a tile and then clicking on a location also can work and is probably easier.
- You do not have to program user recovery from mistakes (tile placement can be final, no backups).

Scrabble reduced

Scrabble is a complex game, and so I'm making a much easier version. If you don't know Scrabble at all, please look it up and/or ask someone (or ask me).

- The board can be smaller than real Scrabble, say 15x15, or even less.
- There is no special scoring on any of the squares on the board. No double-letter, no triple word. But you can add some random ones if you want.
- There are no point values associated with any of the letters.
- Scoring is simply the number of letters played.

- The game needs a list of letters. You can use the real scrabble list available online if you want, or make up your own. Feel free to hard-code this list into your program.
- Each player gets 7 letters to start, and they take turns putting some of them on the board. Each player should only be able to see their own 7 letters, but everyone sees the board.
- In a single turn, the player puts the letters one at a time (by dragging or click,click) and then hits a button that the turn is over. The program should enforce turn taking.
- You do not have to have blank letters if you do not want.
- The game does not have to enforce position rules, like making a player play adjacent to other words or play only in a single line, or spelling. The players themselves can enforce these things, the same as you would do playing with a real board and wooden letters.
- After a player takes their turn, their rack of 7 should re-fill with letters from the list/bag. Both players should draw from the same bag, so you will need communication about what is left in the bag in both host and client (although the program should not SHOW this communication, one person's letters to the other user, or the bag to anyone).
- You do not have to implement "pass" (although that would be pretty easy).
- You do not have to implement whatever end-of-game rules there may be.

That's about as stripped as I can make it. This is just, move some letter from your rack to the board and click to the other person so they can see.

Grading

interface -- can the user see what is going on?	25
game flow - do the controls let you do the right things?	25
communication - does the other side properly reflect what the one side is doing?	25
program structure and comments. I'm counting this for a lot, so that you'll make a big effort to keep your code in order.	25
extra credit is available if you add features (like, some of the ones I took out).	?

submission

github is preferred, but you can zip and submit files to BrightSpace as needed.